

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL-1 EXPERIMENTS

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	C03	Assessment:	ASSESSMENT TOOL 1EXPERIMENTS
Weightage:	40%	Total Marks:	25
C03Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	LOGESH P	Reg.No.:	192521351
Department:	B TECH IT	Date:	10/08/2026

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
 2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
 3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
 4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
 5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.
-

Question 1: Register Transfer Operations and ALU Functionality Analysis

1.1 System Architecture and Register Set Definition

To simulate register transfer microoperations and Arithmetic Logic Unit (ALU) processing, an 8-bit synchronous CPU datapath model is designed using hardware simulation tools (e.g., Logisim Evolution / Verilog HDL). The system core integrates four general-purpose registers ($R0, R1, R2, R3$), an Accumulator (AC), an Instruction Register (IR), and a Status/Flag Register ($FLAGS$). System timing is synchronized via a single master clock signal CLK .

Component Specifications:

- Registers $R0, R1, R2, R3$ (8-bit): General-purpose registers configured with parallel load inputs (LD) and tri-state bus enable drivers (EN).
- Accumulator (AC , 8-bit): Dedicated hardware register connected directly to the output of the ALU.
- Flag Register ($FLAGS$, 4-bit): Latching register storing Zero (Z), Carry (C), Sign (S), and Overflow (V) flags.
- Internal Bus Architecture: Dual 8-bit internal buses (Bus_A and Bus_B) for simultaneous operand delivery.
- Control Unit (CU): Hardwired finite state machine generating control signals at each clock pulse cycle T_n .

1.2 Register Transfer Logic and Microoperations

The hardware executes arithmetic and logical transformations by placing source register contents onto Bus_A and Bus_B , configuring the ALU control lines (S_2, S_1, S_0), and enabling the destination register load gate on the rising edge of CLK .

ALU Operations Matrix:

$S = 000_2$: ADD ($Out = A + B$)

$S = 001_2$: SUB ($Out = A + \bar{B} + 1$)

$S = 010_2$: AND ($Out = A \wedge B$)

$S = 011_2$: OR ($Out = A \vee B$)

1.3 Execution Trace Sequence

Initial Setup:

- $R0 = 0x25$ ($00100101_2 / 37_{10}$)
- $R1 = 0x19$ ($00011001_2 / 25_{10}$)
- $R2 = 0x00, R3 = 0x00, AC = 0x00, FLAGS = [Z = 0, C = 0, S = 0, V = 0]$

Cycle	RTL Operation	ALU Function	Bus-A / Bus-B	Result Output	FLAGS (Z,C,S,V)
T1	$R2 \leftarrow R0$	NOP	Bus-A = 0x25	$R2 = 0x25$	[0, 0, 0, 0]
T2	$AC \leftarrow R0 + R1$	ADD (000)	A=0x25, B=0x19	$AC = 0x3E$ (62)	[0, 0, 0, 0]
T3	$R3 \leftarrow AC$	NOP	Bus-A = 0x3E	$R3 = 0x3E$	[0, 0, 0, 0]
T4	$AC \leftarrow R0 - R1$	SUB (001)	A=0x25, B=0x19	$AC = 0x0C$ (12)	[0, 0, 0, 0]
T5	$R2 \leftarrow R0 \text{ AND } R1$	AND (010)	A=0x25, B=0x19	$R2 = 0x01$ (1)	[0, 0, 0, 0]

1.4 Step-by-Step Mathematical Verification

1. Cycle T_1 ($R2 \leftarrow R0$):

Tri-state buffer for $R0$ asserts $0x25$ onto Bus_A . On clock edge T_1 , load line $R2_{LD}$ latches $0x25$ into $R2$.

2. Cycle T_2 (Addition: $AC \leftarrow R0 + R1$):

$$R0 = 00100101_2 (37_{10})$$

$$R1 = 00011001_2 (25_{10})$$

$$\text{Sum} = 00111110_2 (0x3E/62_{10})$$

No carry-out ($C = 0$), non-zero result ($Z = 0$), MSB is 0 ($S = 0$), no signed overflow ($V = 0$). AC updates to $0x3E$.

3. Cycle T_3 ($R3 \leftarrow AC$):

Contents of AC ($0x3E$) are driven onto Bus_A and latched into $R3$.

4. Cycle T_4 (Subtraction: $AC \leftarrow R0 - R1$):

Two's complement of $R1$: $\overline{R1} + 1 = 11100110_2 + 1 = 11100111_2 (0xE7)$.

$$R0 = 00100101_2$$

$$+(\overline{R1} + 1) = 11100111_2$$

$$\backslash hline \text{Raw Sum} = 1000001100_2 \rightarrow \text{Truncated 8-bit: } 00001100_2 (0x0C/12_{10})$$

Result matches $37 - 25 = 12$.

5. Cycle T_5 (Bitwise AND: $R2 \leftarrow R0 \wedge R1$):

$$00100101_2 (0x25)$$

$$\wedge 00011001_2 (0x19)$$

$$\backslash hline = 00000001_2 (0x01)$$

Destination register $R2$ updates to $0x01$.

1.5 Role of the ALU in Processing Data

The ALU is a purely combinational logic block containing full adders, multiplexers, and logic gates. It does not store state independently; rather, its output dynamically updates as operands arrive on Bus_A and Bus_B . Synchronous execution is achieved because register update lines (LD) latch ALU outputs only on precise positive clock transitions, preventing transient propagation glitches from corrupting processor state.

Question 2: Single-Bus vs. Multi-Bus Organization and Transfer Analysis

2.1 System Topology Comparison

- **Single-Bus System:** A single shared central bus interconnects all registers, the ALU, Memory Address/Data Registers (MAR/MDR), and I/O ports. At any given clock cycle, only ONE data transfer can occur. Multi-operand ALU operations require intermediate holding latches and multiple serial transfer cycles.
- **Multi-Bus System (Three-Bus Topology):** Utilizes dedicated source buses (Bus-A, Bus-B) and a destination bus (Bus-C), alongside independent Memory and I/O buses. Operands are read concurrently from two source registers, processed by the ALU, and written back to a destination register in a single cycle without bus contention.

2.2 System Modeling & Scenario Execution

Cycle	Single-Bus Architecture Operations	Multi-Bus Architecture Operations	Parallelism Advantage
C1	MAR <- 100	MAR <- 100 R2 <- Read I/O Port	Concurrent Memory & I/O setup
C2	MDR <- Memory[MAR]	MDR <- Memory[MAR]	Parallel Memory read cycle
C3	R1 <- MDR	R1 <- MDR	R1 load completed
C4	R2 <- Read I/O Port	R3 <- R1 + R2 MAR <- 102	Simultaneous ALU compute & MAR update
C5	Bus <- R1, ALU_A <- Bus	Memory[102] <- R3 (MDR <- R3)	Memory write completed
C6	Bus <- R2, ALU_B <- Bus, Compute	Execution Complete (5 Cycles)	Multi-Bus finishes in 5 cycles

2.3 Quantitative Time & Efficiency Evaluation

- **Single-Bus Time (T_{single}):** $N_{\text{single}} = 11$ cycles. $T_{\text{single}} = 11 * T_{\text{clk}}$.
- **Multi-Bus Time (T_{multi}):** $N_{\text{multi}} = 5$ cycles. $T_{\text{multi}} = 5 * T_{\text{clk}}$.
- **Speedup Factor (S):** $S = T_{\text{single}} / T_{\text{multi}} = 11 / 5 = 2.20x$ (220% faster execution).
- **Time Reduction Efficiency (E):** Efficiency Gain = $((11 - 5) / 11) * 100\% = 54.55\%$ latency reduction.

2.4 Bottleneck Mitigation Analysis

The single-bus structure introduces severe resource contentions. The ALU cannot execute an addition directly from registers because both operands must share the

same bus, requiring temporary storage latches (ALU_A , ALU_B , AC) and adding 4 cycles of overhead.

In contrast, the multi-bus architecture completely eliminates bus multiplexing overhead. Bus_A and Bus_B feed operands $R1$ and $R2$ into the ALU concurrently, while Bus_C carries the result directly to $R3$ in cycle C_4 . Furthermore, memory address bus paths operate independently of internal register buses, allowing memory write setup ($MAR \leftarrow 102$) to occur simultaneously with computation.

Question 3: 5-Stage Instruction Pipeline Simulation and Performance Evaluation

3.1 5-Stage Pipeline Architecture Overview

The classic 5-stage RISC instruction pipeline divides instruction execution into five overlapping processing phases:

- 1. Fetch (IF):*** Fetch instruction from cache/memory using Program Counter (PC); update $PC \leftarrow PC + 4$.
- 2. Decode (ID):*** Decode instruction opcode and read source registers from Register File.
- 3. Execute (EX):*** Execute ALU operations or calculate effective branch/memory addresses.
- 4. Memory (MEM):*** Access data cache for load/store operations (LW/SW).
- 5. Write-back (WB):*** Write computation/memory results back into target registers.

3.2 Simulation Benchmark Workload

An independent 8-instruction RISC sequence is executed on an ideal pipeline (no hazard stalls):

- I_1 : ADD R1, R2, R3***
- I_2 : SUB R4, R5, R6***
- I_3 : AND R7, R8, R9***
- I_4 : OR R10, R11, R12***
- I_5 : XOR R13, R14, R15***
- I_6 : ADD R16, R17, R18***
- I_7 : SUB R19, R20, R21***
- I_8 : OR R22, R23, R24***

3.3 Pipeline Execution Space-Time Matrix

Inst	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
I1	IF	ID	EX	MEM	WB	-	-	-	-	-	-	-
I2	-	IF	ID	EX	MEM	WB	-	-	-	-	-	-
I3	-	-	IF	ID	EX	MEM	WB	-	-	-	-	-
I4	-	-	-	IF	ID	EX	MEM	WB	-	-	-	-
I5	-	-	-	-	IF	ID	EX	MEM	WB	-	-	-
I6	-	-	-	-	-	IF	ID	EX	MEM	WB	-	-
I7	-	-	-	-	-	-	IF	ID	EX	MEM	WB	-
I8	-	-	-	-	-	-	-	IF	ID	EX	MEM	WB

3.4 Performance Quantification and Mathematical Evaluation

Simulation Parameters:

- Number of instructions (N) = 8
- Pipeline stages (k) = 5
- Clock cycle time (τ) = 2 ns (Clock Frequency f = 500 MHz)

Non-Pipelined System Calculations:

- Execution time per instruction = $k \times \tau = 5 \times 2 \text{ ns} = 10 \text{ ns}$
- Total Non-Pipelined Execution Time (T_{np}):

$$T_{np} = N \times k \times \tau = 8 \times 5 \times 2 \text{ ns} = 80 \text{ ns}$$

- Non-Pipelined Throughput (TP_{np}):

$$TP_{np} = \frac{N}{T_{np}} = \frac{8}{80 \text{ ns}} = 0.1 \text{ inst/ns} = 100 \text{ MIPS}$$

Pipelined System Calculations:

- Total Clock Cycles ($Cycles_{pipe}$) = $k + (N - 1) = 5 + (8 - 1) = 12 \text{ cycles}$
- Total Pipelined Execution Time (T_p):

$$T_p = Cycles_{pipe} \times \tau = 12 \times 2 \text{ ns} = 24 \text{ ns}$$

- Pipelined Throughput (TP_p):

$$TP_p = \frac{N}{T_p} = \frac{8}{24 \text{ ns}} = 0.3333 \text{ inst/ns} = 333.33 \text{ MIPS}$$

Performance Gains:

1. Speedup Factor (S):

$$S = \frac{T_{np}}{T_p} = \frac{80 \text{ ns}}{24 \text{ ns}} = 3.333 \times$$

2. Theoretical Maximum Speedup (S_{max}):

$$S_{max} = k = 5.0 \times$$

3. Pipeline Efficiency (E):

$$E = \frac{S}{k} = \frac{3.333}{5} = 0.6667 \text{ (66.67\%)}$$

Note: As $N \rightarrow \infty$, pipeline speedup approaches the theoretical limit $S \rightarrow k = 5 \times$, and efficiency $E \rightarrow 100\%$.

Question 4: Pipeline Hazard Analysis and Mitigation Techniques

4.1 Hazard Classifications

1. **Structural Hazards:** Hardware resource conflicts where two pipeline stages attempt to access the same physical resource concurrently (e.g., single-ported memory accessed simultaneously by IF and MEM).
2. **Data Hazards:** Occur when an instruction depends on the result of a preceding instruction still in flight within the pipeline:
 - *Read-After-Write (RAW):* True data dependency.
 - *Write-After-Read (WAR):* Anti-dependency.
 - *Write-After-Write (WAW):* Output dependency.
3. **Control Hazards:** Branch and jump instructions that alter the *PC* value, introducing uncertainty during instruction fetch phases.

4.2 Benchmark Hazard Test Sequence

I1: ADD R1, R2, R3 ; Writes destination R1 in WB stage
I2: SUB R4, R1, R5 ; RAW Hazard on R1 (produced by I1)
I3: LW R6, 0(R4) ; Memory Load to R6; RAW Dependency on R4 (from I2)
I4: AND R7, R6, R8 ; Load-Use Data Hazard on R6 (from I3)
I5: BEQ R7, R0, TARGET ; Control Hazard (Branch dependent on R7 from I4)

4.3 Mitigation Techniques Mechanics

- **Data Forwarding (Bypassing):** Hardware paths route calculated outputs directly from pipeline registers (EX/MEM or MEM/WB) back to the EX stage multiplexer inputs, bypassing register file write delays.
- **Load-Use Stall Insertion:** Because data loaded via LW is not available until the end of the MEM stage, an instruction immediately consuming that loaded register requires a hardware stall bubble in the ID stage.

- **Branch Prediction & Target Buffers (BTB):** Dynamic 2-bit saturating prediction counters predict branch direction in the IF stage. If correct, zero penalty cycles are incurred; if mispredicted, speculative instructions are flushed, incurring a 2-cycle penalty.

4.4 Hazard Pipeline Timing Matrix (Mitigated Execution)

Inst	C1	C2	C3	C4	C5	C6	C7	C8	Applied Mitigation Mechanism
I1: ADD	IF	ID	EX	MEM	WB	-	-	-	Base instruction
I2: SUB	-	IF	ID	EX	MEM	WB	-	-	EX-to-EX Forwarding for R1 from I1
I3: LW	-	-	IF	ID	EX	MEM	WB	-	EX-to-EX Forwarding for R4 from I2
I4: AND	-	-	-	IF	ID	STALL	EX	MEM	1-Cycle Stall for Load-Use on R6
I5: BEQ	-	-	-	-	IF	ID	EX	MEM	BTB Prediction + MEM-to-EX Bypass

4.5 Quantitative Performance Impact Analysis

Cycle Comparison:

- **Unmitigated System Execution (Stalls only):**
 - I_2 stalls 2 cycles waiting for I_1 Write-Back.
 - I_4 stalls 2 cycles waiting for I_3 Write-Back.
 - I_5 stalls 2 cycles for branch resolution.
 - Total Cycles = 13 Clock Cycles ($IPC = \frac{5}{13} = 0.38$)
- **Mitigated System Execution (Forwarding + BTB):**
 - I_2 stalls 0 cycles (Forwarded).
 - I_4 stalls 1 cycle (Load-Use constraint).
 - I_5 stalls 0 cycles (Correct Branch Prediction).
 - Total Cycles = 9 Clock Cycles ($IPC = \frac{5}{9} = 0.56$)

Performance Gain:

$$\text{Cycle Reduction Efficiency} = \left(\frac{13 - 9}{13} \right) \times 100\% = 30.77\% \text{ reduction in execution latency}$$

$$\text{Throughput Improvement} = \left(\frac{0.56 - 0.38}{0.38} \right) \times 100\% = 47.37\% \text{ IPC gain}$$

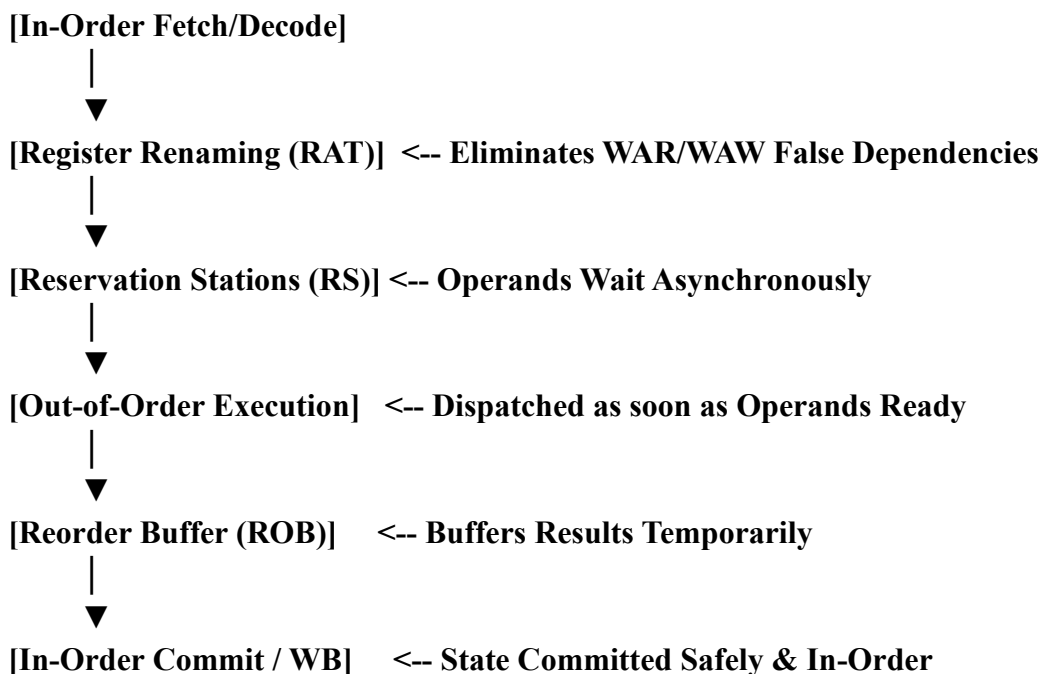
Question 5: Superscalar, Out-of-Order Execution, Speculation, and SMT Architecture

5.1 Superscalar Processing Principles

Superscalar architectures exploit Instruction-Level Parallelism (ILP) by deploying multiple parallel execution pipelines (multiple ALUs, FPUs, Load/Store Units) capable of fetching, decoding, issuing, and executing N instructions per clock cycle.

5.2 Out-of-Order Execution (OoO) and Speculation Paradigm

To overcome pipeline stalls from data dependencies and memory cache misses, high-performance processors utilize dynamic Out-of-Order (OoO) execution pipelines:



1. **Register Alias Table (RAT / Renaming):** Maps architectural registers ($R0--R31$) to a larger physical register file pool, eliminating false dependencies: Write-After-Read (WAR) and Write-After-Write (WAW).
2. **Reservation Stations (RS):** Issue queues where instructions wait asynchronously. An instruction dispatches to an execution unit as soon as its required input operands become available, regardless of original program order.

3. **Reorder Buffer (ROB) & Precise Exceptions:** Results are computed Out-of-Order but stored temporarily in the ROB. Instructions commit state changes to architectural registers strictly In-Order, preserving precise exception handling and rollback capability upon branch misprediction.
4. **Speculative Execution:** Branch prediction units allow instructions following a branch to be fetched and executed speculatively. If the branch resolves correctly, ROB commits the speculative results; if mispredicted, speculative pipeline state is flushed without affecting memory or architectural registers.

5.3 Simultaneous Multithreading (SMT) and Hyper-Threading

While superscalar architectures focus on Instruction-Level Parallelism (ILP) within a single thread, Simultaneous Multithreading (SMT) combines ILP with Thread-Level Parallelism (TLP).

In an SMT architecture (such as Intel Hyper-Threading):

- **Duplicated Hardware Resources:** Architectural state components are replicated per logical thread (Program Counters, Register Files, Control Logic, Interrupt Controllers).
- **Shared Hardware Resources:** Core execution hardware is shared (Execution Units/ALUs, Reservation Stations, L1/L2 Caches, Translation Lookaside Buffers).

SMT enables instructions from multiple independent software threads to enter execution pipelines concurrently in the same clock cycle, converting unused pipeline capacity into productive throughput.

5.4 Architectural Paradigm Comparison

Feature	Scalar Core	In-Order Superscalar	Out-of-Order Superscalar	SMT / Hyper-Threading
Peak Issue Rate	1 inst/cycle	N inst/cycle (N ≥ 2)	N inst/cycle (N ≥ 4)	N inst/cycle across M threads
Execution Order	Strict In-Order	Strict In-Order	Out-of-Order (OoO)	Out-of-Order across threads
Resource Utilization	Low (20-30%)	Moderate (40-50%)	High (60-70%)	Exemplary (85-95%)
Silicon Area Cost	Baseline (1.0x)	Moderate (~1.2x)	High (~1.4x)	Optimal (~1.45x for +35% IPC)

5.5 Quantitative Throughput and Resource Utilization Analysis

Hardware Benchmark Context:

Quad-issue core containing 4 execution ALUs/Functional Units.

Cycle Slot Occupancy Illustration:

Scalar In-Order Core:

[Cycle 1] [Inst 1] [Idle] [Idle] [Idle] --> 25% Unit Utilization
[Cycle 2] [STALL] [STALL] [STALL] [STALL] --> Cache Miss Stall

Superscalar Out-of-Order Core (Single Thread):

[Cycle 1] [Inst 1] [Inst 2] [Inst 3] [Idle] --> 75% Unit Utilization
[Cycle 2] [Inst 4] [STALL] [STALL] [STALL] --> Dependency Stall

Superscalar SMT Core (Dual Thread: Thread A + Thread B):

[Cycle 1] [ThA_I1] [ThA_I2] [ThB_I1] [ThB_I2] --> 100% Unit Utilization
[Cycle 2] [ThA_ST] [ThB_I3] [ThB_I4] [ThB_I5] --> 75% Utilization (Thread B fills Thread A stall)

Performance Comparison Metrics:

1. Single-Threaded In-Order Core:

- Average Instructions Per Cycle (IPC) = 1.2
- Functional Unit Utilization = $\frac{1.2}{4} \times 100\% = 30\%$

2. Single-Threaded Out-of-Order Core:

- Bypasses data stalls using RS and ROB queues.
- Average IPC = 2.6
- Functional Unit Utilization = $\frac{2.6}{4} \times 100\% = 65\%$

3. Dual-Thread SMT Core:

- When Thread A incurs a cache miss or long-latency operation, Thread B immediately fills the vacant execution slots in the exact same clock cycle.
- Combined System IPC = 3.6
- Functional Unit Utilization = $\frac{3.6}{4} \times 100\% = 90\%$

Conclusion:

SMT eliminates both horizontal waste (unused execution slots in a cycle) and vertical waste (cycles lost entirely to memory latency stalls), maximizing processor instruction throughput and resource utilization.

Code of Conduct:

I LOGESH P / 192521351 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P Logesh

MARKS ALLOCATION

	Understanding of Concepts(20%)	Experimental Design(20%)	Use of Tools(25%)	Analysis of Results(20%)	Documentation(15 %)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation